

WEEK 8: MAXIMUM FLOW

Advanced Algorithms

ROADMAP

1. The Max-Flow Problem
2. Ford-Fulkerson and Residual Graphs
3. Max-Flow = Min-Cut
4. Edmonds-Karp
5. Applications: Bipartite Matching, Matrix Rounding

THE MAX-FLOW PROBLEM

A PUZZLE TO KEEP IN MIND

Round every entry of this matrix to an integer (here: 0 or 1) so that **every row sum and every column sum is unchanged**.

Row and column sums are already integers — but is such a rounding always possible?

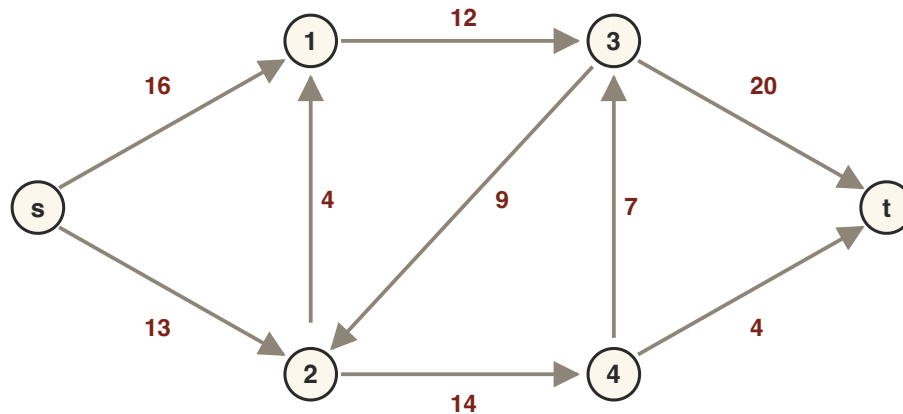
0.7	0.8	0.5	2
0.3	0.1	0.6	1
1	0.1	0.9	2
2	1	2	

Red: row and column sums.

By the end of today we will solve this with a **maximum-flow computation**.

FLOW NETWORKS

SETTING A directed graph $G = (V, E)$ with a *capacity* $c(e) \geq 0$ on each edge, a *source* s and a *sink* t .



Edge labels are capacities: how much can flow along each edge.

Think of routing water, traffic, or data from s to t .

ST-FLOWS

DEFINITION An **st-flow** assigns a value $f(e) \geq 0$ to each edge such that:

- *Capacity*: $f(e) \leq c(e)$ for every edge e ,
- *Conservation*: for every vertex $v \neq s, t$:

$$\sum_{e \in \delta^-(v)} f(e) = \sum_{e \in \delta^+(v)} f(e)$$

$\delta^-(v)$ = edges entering v , $\delta^+(v)$ = edges leaving v . Flow in equals flow out, except at the source and the sink.

One valid flow always exists: the zero flow $f \equiv 0$. We want the opposite extreme.

THE VALUE OF A FLOW

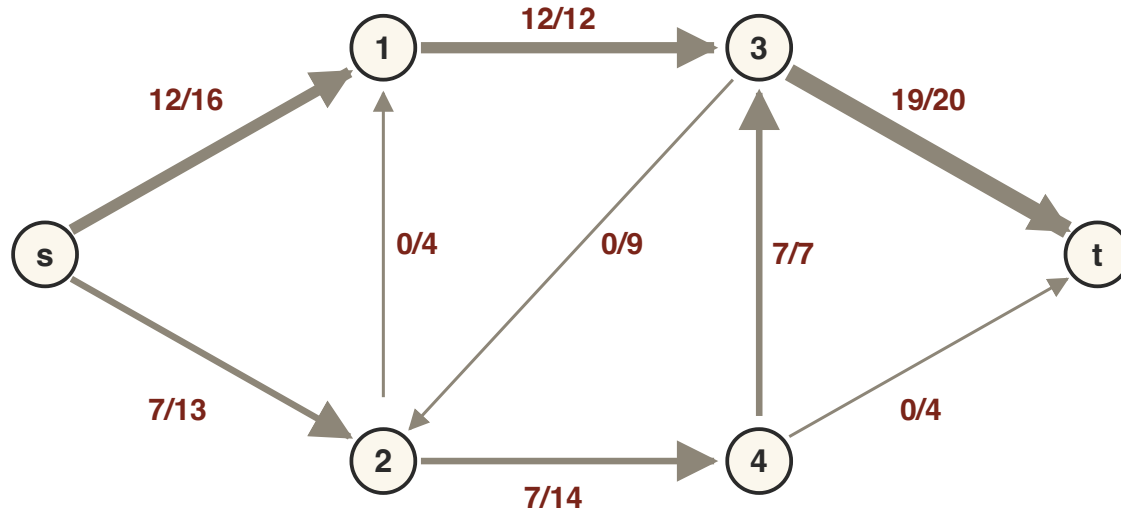
DEFINITION The **value** of f is the net flow out of the source:

$$|f| = \sum_{e \in \delta^+(s)} f(e) - \sum_{e \in \delta^-(s)} f(e).$$

PROBLEM **Maximum flow:** find an st-flow of maximum value.

By conservation, everything leaving s must arrive at t (we will prove a sharp version of this later: the value equals the net flow across *any* st-cut).

EXAMPLE: A FLOW OF VALUE 19



Labels are *flow/capacity*. Check conservation at each internal vertex.

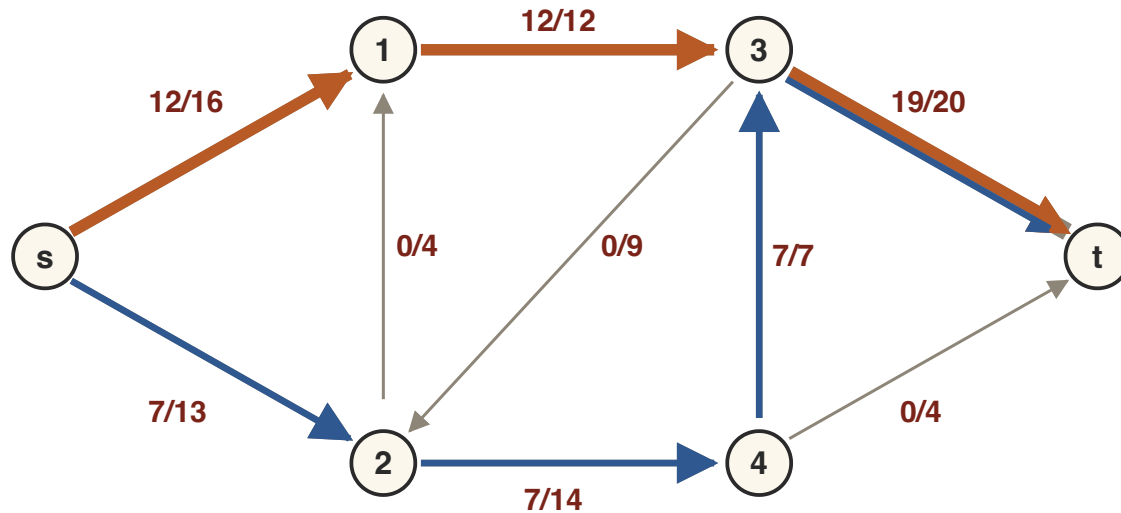
$|f| = 12 + 7 = 19$. Is this the best possible? How would we even know?

FLOW DECOMPOSITION

THEOREM Any st-flow f can be written as a sum of flows along at most $|E|$ st-paths and cycles.

A flow can also contain cycles — flow going round in circles — which contribute nothing to the value.

FLOW DECOMPOSITION



Our value-19 flow decomposes as

- 12 units on the path $s \rightarrow 1 \rightarrow 3 \rightarrow t$,
- 7 units on the path $s \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow t$.

FLOW DECOMPOSITION: PROOF SKETCH

Phase 1. While $|f| > 0$: starting at s , follow positive-flow edges — conservation means we never get stuck before reaching t . Subtract the bottleneck value of the st -path found.

Phase 2. When $|f| = 0$ but some edge still carries flow: following positive-flow edges from it must eventually revisit a vertex — a *cycle*. Subtract its bottleneck.

FLOW DECOMPOSITION: PROOF SKETCH

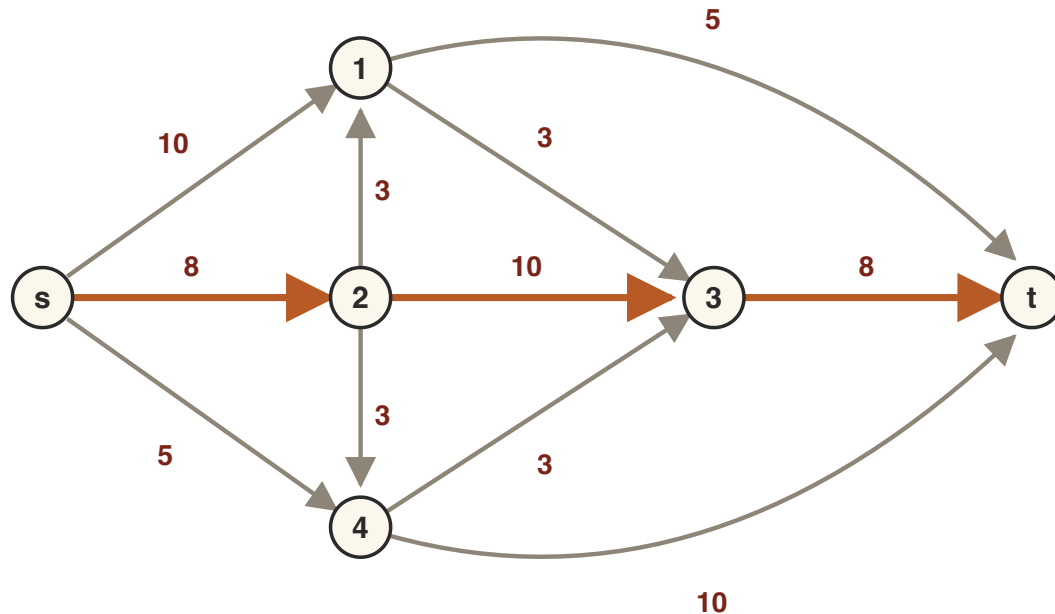
Each step zeroes out at least one edge, so there are at most $|E|$ steps. ■

TAKEAWAY Flows look complicated, but they are just paths plus cycles. Max flow is really about finding good path packings.

FORD-FULKERSON AND RESIDUAL GRAPHS

A GREEDY ATTEMPT

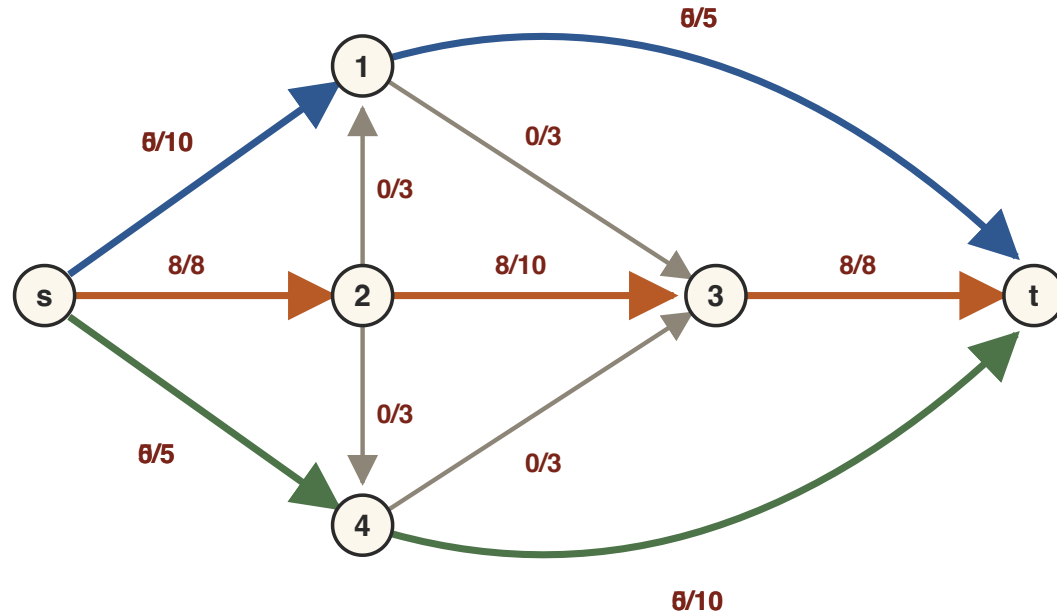
Natural idea: repeatedly find an st -path with spare capacity and push as much flow as possible along it. Start with the **widest** path: the one whose *bottleneck*, its smallest edge capacity, is largest.



Widest path: $s \rightarrow 2 \rightarrow 3 \rightarrow t$ with bottleneck 8.

GREEDY GETS STUCK AT 18

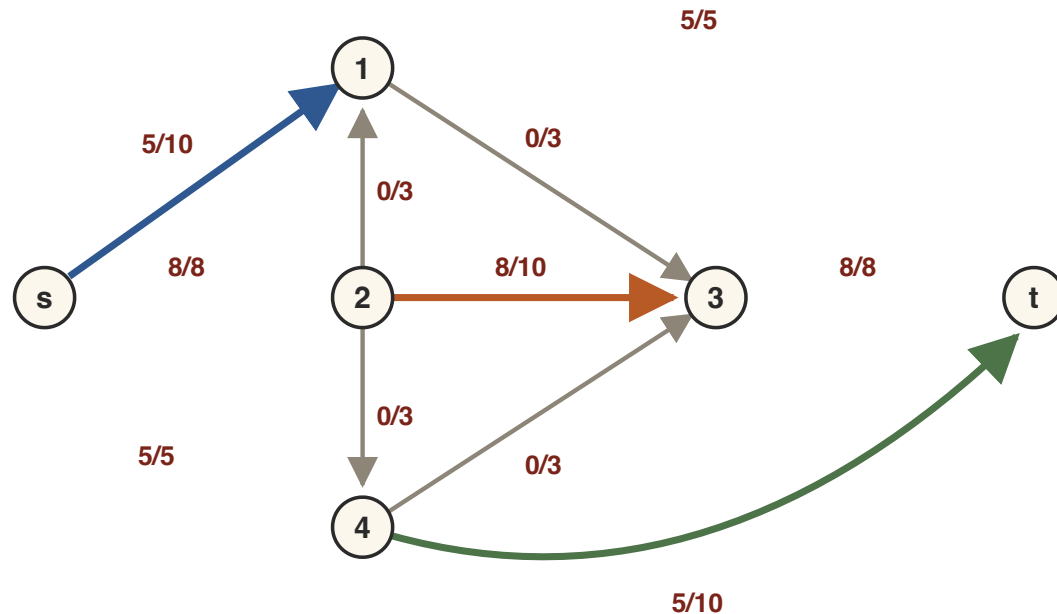
Continue: push 5 along $s \rightarrow 1 \rightarrow t$, then 5 along $s \rightarrow 4 \rightarrow t$.



$|f| = 8 + 5 + 5 = 18$. Is there another st -path with spare capacity?

GREEDY GETS STUCK AT 18

Four edges are **saturated**, $f(e) = c(e)$: they can carry nothing more, so remove them.



From s we can still reach 1 and 3, but not t : no st -path is left. Greedy stops at $|f| = 18$.

BUT 18 IS NOT OPTIMAL

CAREFUL The maximum flow in this network has value 21. Greedy path-packing can get stuck at a suboptimal flow — early paths make commitments that later paths cannot work around.

What went wrong: the 8 units on edge $2 \rightarrow 3$ hog capacity that is needed on $3 \rightarrow t$; vertex 4 would happily take flow from 2 instead.

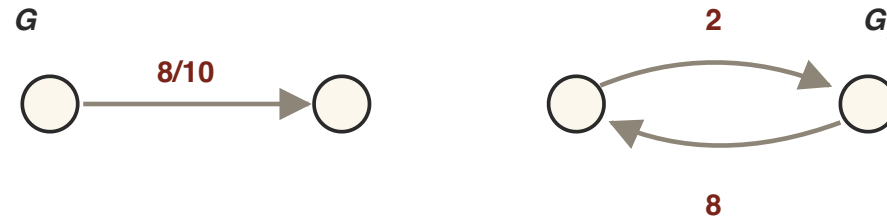
The fix: allow a new path to decrease flow on an edge, i.e. to undo earlier decisions.

The bookkeeping device for this is the *residual network*.

THE RESIDUAL NETWORK

- DEFINITION** Given a flow f in G , the residual network G_f has the same vertices, and for each edge $e = (u, v)$ of G :
- a *forward edge* (u, v) with residual capacity $c(e) - f(e)$, if positive;
 - a *backward edge* (v, u) with residual capacity $f(e)$, if positive.

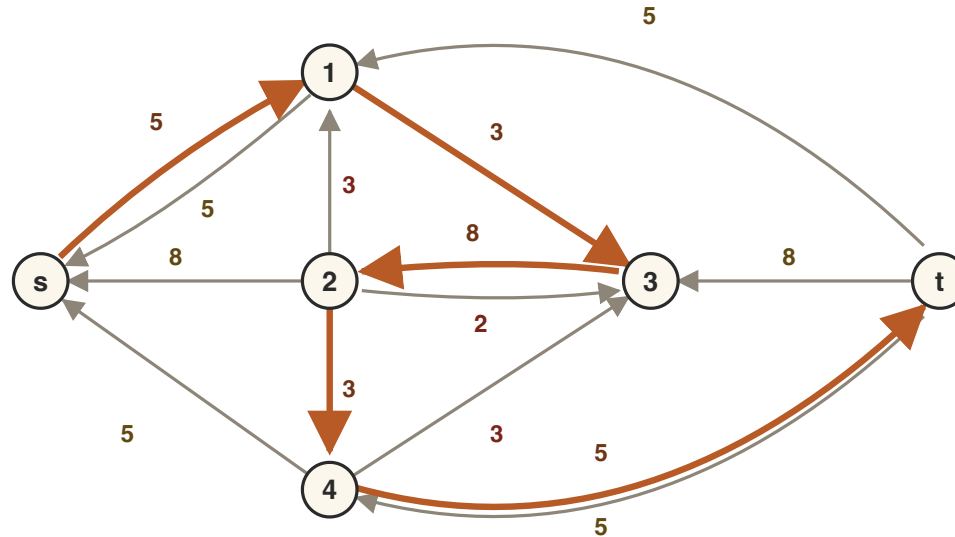
THE RESIDUAL NETWORK



An edge carrying 8 of 10 units becomes: 2 more can be pushed forward, 8 can be pushed back (undone).

G_f has at most $2|E|$ edges — same order of size as G .

RESIDUAL NETWORK OF THE GREEDY FLOW

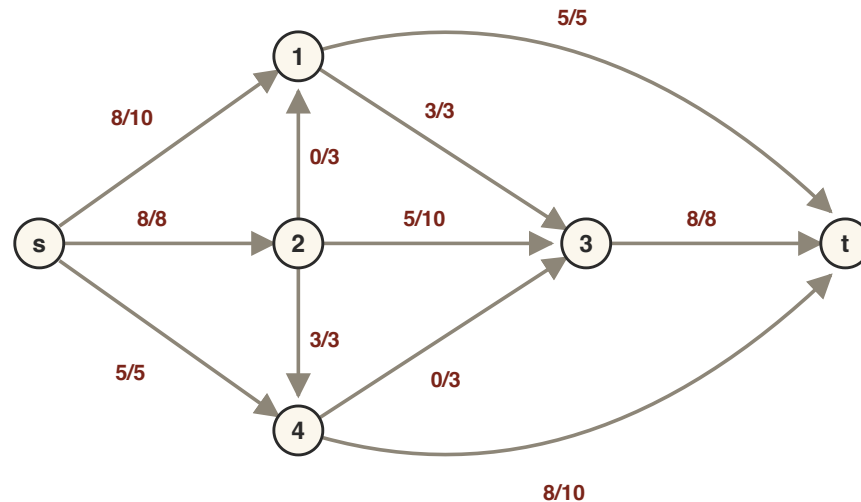


The residual network G_f of the stuck greedy flow.
Orange: an st -path in G_f ! It uses the *backward* edge $3 \rightarrow 2$.

Path $s \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow t$ has residual capacity $\min(5, 3, 8, 3, 5) = 3$.

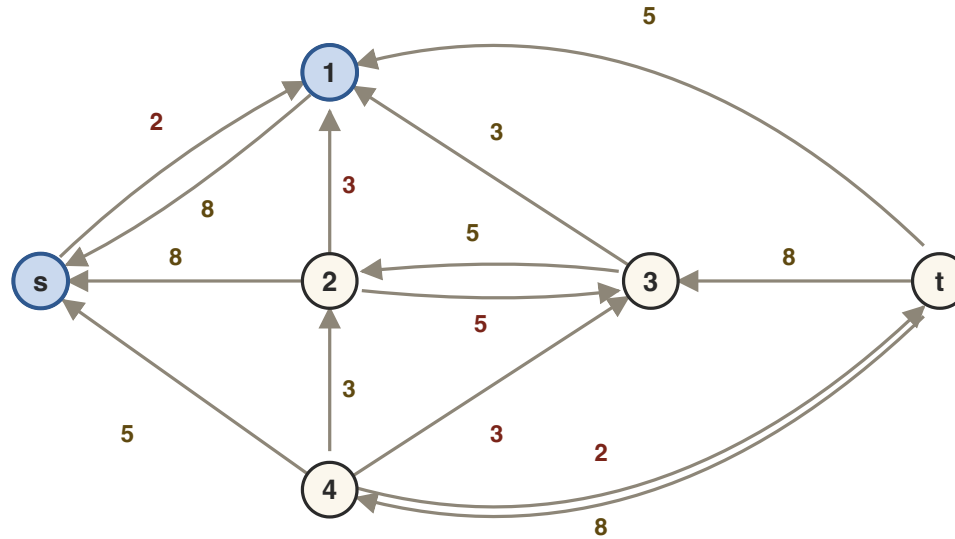
AUGMENTING TO VALUE 21

Push 3 units along the path: forward edges gain 3 units of flow, the backward edge $3 \rightarrow 2$ removes 3 units from edge $2 \rightarrow 3$.



The new flow: still feasible, still conserving, and $|f| = 8 + 8 + 5 = 21$.

THE NEW RESIDUAL NETWORK



G_f after augmenting to 21.

Only one residual edge leaves s , and from 1 the only way out leads back to s : reachable from s is just $\{s, 1\}$.

So there is **no** st -path in G_f . Coincidence that this means we are optimal? Not at all — that is the next part.

AUGMENTING PATHS

DEFINITION An **augmenting path** for a flow f is an st-path P in the residual network G_f . Its *residual capacity* $c_f(P)$ is the minimum residual capacity of its edges.

AUGMENTING PATHS

KEY FACT If f has an augmenting path P , then pushing $c_f(P)$ units along P (add on forward edges, subtract on backward edges) yields a valid flow of value

$$|f| + c_f(P) > |f|.$$

Feasibility check: forward edges stay $\leq c(e)$ by definition of residual capacity; backward edges stay ≥ 0 ; conservation holds because each internal path vertex gets $+$ and $-$ adjustments that cancel.

THE FORD-FULKERSON METHOD

```
FordFulkerson(G, s, t, c):  
    initialise  $f(e) = 0$  for all  $e$            // start from the zero flow  
    while there is an st-path  $P$  in  $G_f$ :  
        choose an augmenting path  $P$   
        augment  $f$  by  $c_f(P)$  along  $P$        // value strictly increases  
    return  $f$ 
```

Called a *method* rather than an algorithm: it does not specify *which* augmenting path to choose. Different choices give different running times.

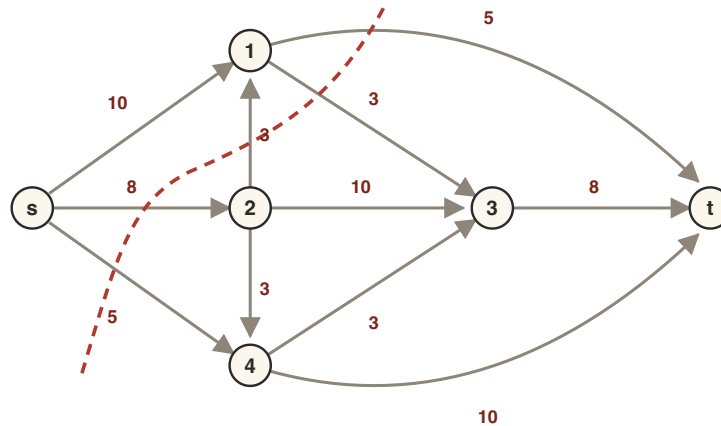
Two questions we must answer:

- Does the loop terminate?
- When it stops (no augmenting path), is f actually a *maximum* flow?

MAX-FLOW = MIN-CUT

ST-CUTS

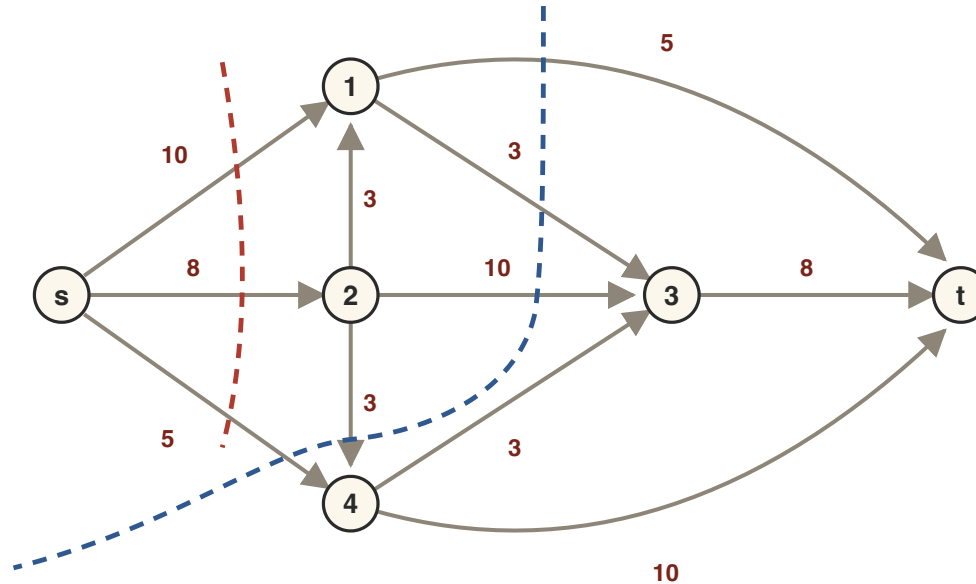
DEFINITION An **st-cut** is a partition (X, Y) of the vertices with $s \in X$ and $t \in Y$. Its **capacity** is $\sum_{e \in \delta^+(X)} c(e)$ — the total capacity of edges leaving X .



The dashed cut $X = \{s, 1\}$ has capacity $8 + 5 + 3 + 5 = 21$ (edges $s \rightarrow 2$, $s \rightarrow 4$, $1 \rightarrow 3$, $1 \rightarrow t$).

ST-CUTS

Other examples:



- $X = \{s\}$ has capacity $10 + 8 + 5 = 23$.
- $X = \{s, 1, 2\}$ has capacity $5 + 3 + 3 + 10 + 5 = 26$.

Note: edges *entering* X do not count.

FLOW ACROSS A CUT

FLOW FACT For any flow f and any st-cut (X, Y) :

$$|f| = \sum_{e \in \delta^+(X)} f(e) - \sum_{e \in \delta^-(X)} f(e).$$

The value of the flow equals the net flow crossing *any* st-cut.

FLOW ACROSS A CUT

Proof. Sum the quantity $\sum_{e \in \delta^+(v)} f(e) - \sum_{e \in \delta^-(v)} f(e)$ over all $v \in X$.

- By conservation, every $v \neq s$ contributes 0; the vertex s contributes exactly $|f|$.
- Edges with both endpoints in X appear once with $+$ and once with $-$: they cancel.
- What survives: edges leaving X (counted $+$) and edges entering X (counted $-$). ■

WEAK DUALITY: $\text{FLOW} \leq \text{CUT}$

For every flow f and every st-cut (X, Y) :

$$\begin{aligned} |f| &= \sum_{e \in \delta^+(X)} f(e) - \sum_{e \in \delta^-(X)} f(e) \\ &\leq \sum_{e \in \delta^+(X)} f(e) \leq \sum_{e \in \delta^+(X)} c(e). \end{aligned}$$

COROLLARY $\max \text{ st-flow} \leq \min \text{ st-cut capacity}.$

WEAK DUALITY: $\text{FLOW} \leq \text{CUT}$

So a cut of capacity 21 *certifies* that our flow of value 21 is maximum — cuts are optimality certificates for flows.

Remarkably, a matching cut **always** exists.

THE MAX-FLOW MIN-CUT THEOREM

THEOREM For an st-flow f , the following are equivalent:

1. f is a maximum flow;
2. there is no augmenting path in G_f ;
3. there is an st-cut of capacity exactly $|f|$.

In particular: maximum st-flow value = minimum st-cut capacity .

Plan: $(1) \Rightarrow (2)$ and $(3) \Rightarrow (1)$ are quick; the heart of the proof is $(2) \Rightarrow (3)$.

THE TWO EASY IMPLICATIONS

THEOREM For an st-flow f , the following are equivalent:

1. f is a maximum flow;
2. there is no augmenting path in G_f ;
3. there is an st-cut of capacity exactly $|f|$.

(1) $\Rightarrow$ (2) Contrapositive: if G_f has an augmenting path P , pushing $c_f(P) > 0$ along it gives a strictly larger flow — so f was not maximum.

THE TWO EASY IMPLICATIONS

THEOREM For an st-flow f , the following are equivalent:

1. f is a maximum flow;
2. there is no augmenting path in G_f ;
3. there is an st-cut of capacity exactly $|f|$.

(3) $\Rightarrow$ (1) By weak duality every flow value is at most the capacity of every cut. If some cut has capacity exactly $|f|$, no flow can beat f : it is maximum.

Remaining: **(2) $\Rightarrow$ (3)** — from “no augmenting path” we must *construct* a cut of capacity $|f|$.

(2) $\Rightarrow$ (3): READING A CUT OFF G_f

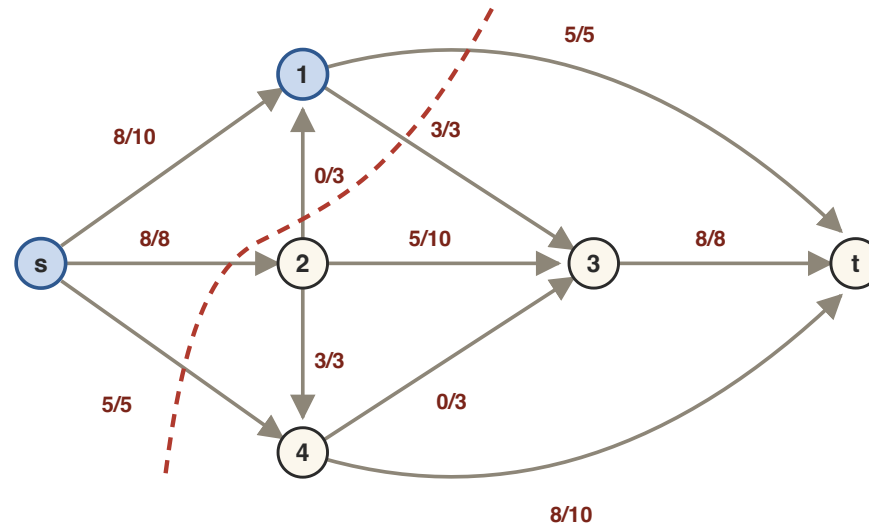
Suppose G_f has no st-path. Let

$X = \{ v : v \text{ reachable from } s \text{ in } G_f \}$ and Y the rest.

- $s \in X$ by definition, and $t \in Y$ since there is no st-path: (X, Y) is an st-cut.
- No residual edge crosses from X to Y — otherwise the head would be reachable too.

(2) $\Rightarrow$ (3): READING A CUT OFF G_f

Our example after augmenting to 21: the vertices reachable from s in G_f were $X = \{s, 1\}$.



Every edge leaving X is saturated (8/8, 5/5, 3/3, 5/5) and the one edge entering X carries 0. Capacity $8 + 5 + 3 + 5 = 21 = |f|$.

(2) $\Rightarrow$ (3): READING A CUT OFF G_f

No residual edge leaves X — in G this says two things:

(A) No *forward* residual edge leaves X : every edge of G leaving X is **saturated**, $f(e) = c(e)$.

(B) No *backward* residual edge leaves X : every edge of G entering X carries **zero flow**, $f(e) = 0$.

In our example: after augmenting to 21, the vertices reachable from s in G_f are exactly $\{s, 1\}$ — the dashed cut from before, with capacity 21.

(2) $\Rightarrow$ (3): FINISHING THE COMPUTATION

Apply the Flow Fact to the cut (X, Y) we just built:

$$\begin{aligned} |f| &= \sum_{e \in \delta^+(X)} f(e) - \sum_{e \in \delta^-(X)} f(e) \\ &\stackrel{(b)}{=} \sum_{e \in \delta^+(X)} f(e) \stackrel{(a)}{=} \sum_{e \in \delta^+(X)} c(e). \end{aligned}$$

So the capacity of (X, Y) is exactly $|f|$ — statement (3). ■

(2) $\Rightarrow$ (3): FINISHING THE COMPUTATION

CONSEQUENCE When Ford-Fulkerson stops, the flow is maximum. And the vertices reachable from s in the final residual network hand us a minimum cut for free .

A SECOND PROOF IS COMING

PREVIEW In **Week 10** we will see a second, completely different proof of $\text{max-flow} = \text{min-cut}$, via **LP duality**: flows and cuts are optimal solutions of a primal-dual pair of linear programs.

Same theorem, two proofs, two generalisations: the residual-graph proof gives algorithms; the LP proof places the theorem inside a much bigger duality landscape.

INTEGRAL CAPACITIES: TERMINATION

THEOREM If all capacities are integers, Ford-Fulkerson terminates, and it outputs a maximum flow that is **integral** (every $f(e)$ is an integer).

Why. Starting from the zero flow, all capacities in G_f stay integral, so each augmentation is by an integer amount, i.e. increases $|f|$ by at least 1. With $F = \text{max-flow value}$, the loop runs at most F times.

INTEGRAL CAPACITIES: TERMINATION

Moreover the flow found is built from integer augmentations, so it is **integral**.

CAREFUL With *irrational* capacities Ford-Fulkerson can actually fail to terminate!

HOW FAST IS FORD-FULKERSON?

RUNTIME Each iteration (find a path in G_f by BFS/DFS, augment) takes $O(|E|)$ time, so on integer capacities Ford-Fulkerson runs in $O(F \cdot |E|)$ time.

Two loose ends:

- The bound depends on the flow *value* F , which can be exponential in the input size — fixed by **Edmonds-Karp** next.
- Integrality looks like a technicality, but it is exactly what our **applications** will exploit.

EDMONDS-KARP

CHOOSING PATHS WISELY

$O(F \cdot |E|)$ is *not* polynomial in the input size: a capacity of 10^9 fits in 30 bits but may force 10^9 augmentations if paths are chosen badly.

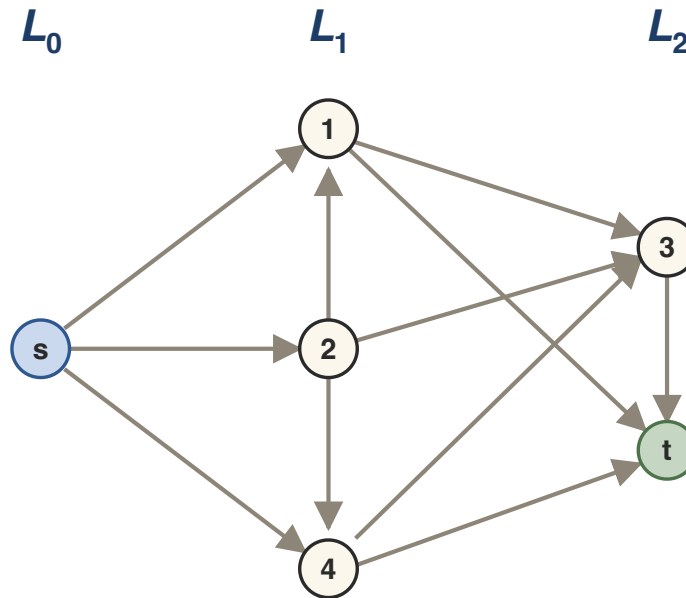
EDMONDS-KARP (1970) Run Ford-Fulkerson, always choosing an augmenting path with the **fewest edges**. The running time is $O(|V| \cdot |E|^2)$ — independent of the capacities.

CHOOSING PATHS WISELY

- A fewest-edges st-path in G_f is found by **BFS** (ignore capacities) in $O(|E|)$ time per round.
- So it suffices to show: the number of rounds is $O(|V| \cdot |E|)$.

BFS LEVELS OF G_f

Fix the current G_f and let L_i be the vertices at distance i from s (BFS layers), with $t \in L_d$.



Levels for our example at $f = 0$: $L_1 = \{1, 2, 4\}$, $L_2 = \{3, t\}$, so $d = 2$.

Every edge of G_f goes at most one level forward (BFS property), so a shortest st-path is $s - v_1 - \dots - v_{d-1} - t$ with $v_i \in L_i$.

DISTANCES NEVER DECREASE

What can one shortest-path augmentation change in G_f ?

- Augmenting on (v_i, v_{i+1}) may *delete* that edge (if saturated) and may *create* the reverse edge (v_{i+1}, v_i) .
- So every **new** edge points from some L_{i+1} back to L_i — one level *backwards*.
- A path using a backward edge wastes at least two steps: adding such edges cannot create an st-path shorter than d .

KEY FACT Under shortest-path augmentation, the st-distance in G_f never decreases.

COUNTING THE ROUNDS

- Each augmentation **saturates** at least one edge of the path (the bottleneck), deleting an $L_i \rightarrow L_{i+1}$ edge from G_f .
- While the distance stays equal to d , shortest paths only use $L_i \rightarrow L_{i+1}$ edges — and new edges never have this form. There are at most $|E|$ such edges: each edge of G yields two opposite residual edges, at most one of which is level-forward. So after at most $|E|$ augmentations, no shortest path of length d remains: the **distance strictly increases**.

COUNTING THE ROUNDS

- The distance is an integer between 1 and $|V| - 1$, so it can increase at most $|V|$ times.
- At most $|E|$ augmentations per distance value, at most $|V|$ distance values.

THEOREM Edmonds-Karp performs $O(|V| \cdot |E|)$ augmentations; with $O(|E|)$ BFS per round the total running time is $O(|V| \cdot |E|^2)$ — for arbitrary real capacities.

PROGRESS ON MAX-FLOW

ALGORITHM	YEAR	TIME
Edmonds-Karp	1970	$O(VE^2)$
Dinitz	1970	$O(V^2E)$
Binary blocking flow (Goldberg-Rao)	1998	$O(E \min\{V^{2/3}, E^{1/2}\} \log \frac{V^2}{E} \log U)$
BLNPSSSW / BLLSSSW	2020	$\tilde{O}((E + V^{3/2}) \log U)$
Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva	2022	$O(E^{1+o(1)} \log U)$

U is the largest (integer) capacity, so $\log U$ is the number of bits per capacity.

Using very different techniques (continuous optimisation, dynamic graph data structures), we now have **nearly optimal** algorithms for max flow — almost linear in the number of edges.

CHECK YOUR UNDERSTANDING

Discuss at your table.

Q1. Given a maximum st-flow, how do you compute a minimum st-cut in $O(|E|)$ time?

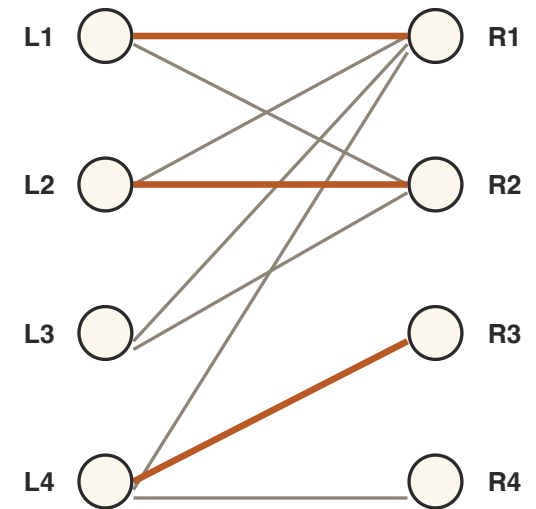
Q2. Given a minimum st-cut, can you compute a maximum st-flow quickly?

APPLICATIONS

APPLICATION 1: MATCHING

DEFINITION A **matching** is a set of edges M such that each vertex touches at most one edge of M . A matching covering *all* vertices is **perfect**.

In Weeks 3-4 we solved maximum matching on *trees* by DP and greedy. In a **bipartite** graph (parts L , R , all edges between) it models assigning jobs to applicants.

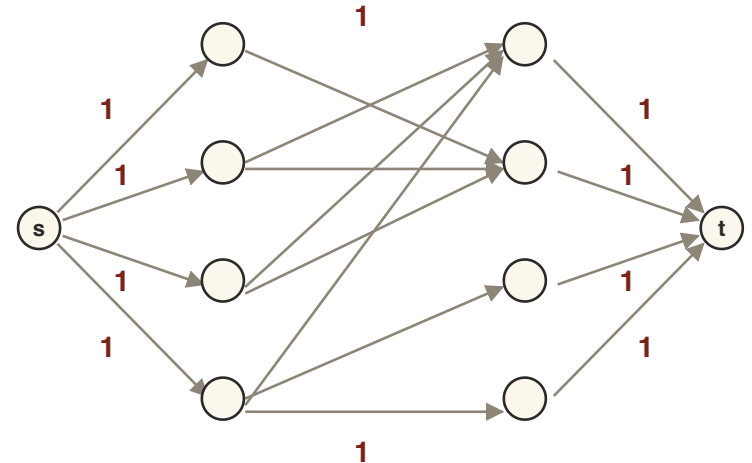


The size of a maximum matching in this graph is 3.

MATCHING AS A FLOW PROBLEM

Build a flow network:

- direct all edges from L to R , capacity 1;
- add source s with edges $s \rightarrow \ell$ of capacity 1 for every $\ell \in L$;
- add sink t with edges $r \rightarrow t$ of capacity 1 for every $r \in R$.



All capacities are 1.

CLAIM max flow value = maximum matching size.

PROOF: $\text{FLOW} \geq \text{MATCHING}$

Take any matching M .

- Each matching edge (ℓ, r) gives a path $s \rightarrow \ell \rightarrow r \rightarrow t$.
- Because M is a matching, these paths are **edge-disjoint** (each ℓ and each r is used once).
- Sending one unit along each path is a valid flow of value $|M|$.

HALF DONE $\max \text{flow} \geq \max \text{matching}$.

PROOF: $\text{FLOW} \leq \text{MATCHING}$ — VIA INTEGRALITY

All capacities are 1, so by the integrality theorem there is a **maximum flow with values in $\{0, 1\}$** .

CLAIM The $L \rightarrow R$ edges carrying flow 1 form a matching of size $|f|$.

PROOF: $\text{FLOW} \leq \text{MATCHING}$ — VIA INTEGRALITY

CLAIM The $L \rightarrow R$ edges carrying flow 1 form a matching of size $|f|$.

- $\ell \in L$ receives at most 1 unit (from s), so by conservation at most one flow-1 edge *leaves* ℓ .
- $r \in R$ can send at most 1 unit (to t), so at most one flow-1 edge *enters* r .

So every vertex touches at most one selected edge: the selected $L \rightarrow R$ edges **form a matching**.

PROOF: $\text{FLOW} \leq \text{MATCHING}$ — VIA INTEGRALITY

- The number of selected edges is the flow crossing the cut $(\{s\} \cup L, R \cup \{t\})$, which is $|f|$. ■

RUNTIME $F \leq |V|$, so plain Ford-Fulkerson already solves bipartite matching in $O(|V| \cdot |E|)$ time. Integrality did the heavy lifting.

KÖNIG'S THEOREM

THEOREM (KÖNIG, 1931) If G is bipartite, then $\nu(G) = \tau(G)$: the maximum matching size equals the minimum vertex cover size.

Weak duality (Week 3): $\nu(G) \leq \tau(G)$ in every graph, since a cover needs a distinct vertex for each edge of a matching.

Week 3 proved the bipartite case with alternating paths. Today we read a cover off a minimum cut of the matching network.

KÖNIG VIA MIN CUT

Plan: turn a minimum cut of the matching network into a vertex cover of the same size, by associating each cut edge with a vertex:

- cutting $s \rightarrow \ell$ costs 1 $\Leftrightarrow$ put ℓ in the cover;
- cutting $r \rightarrow t$ costs 1 $\Leftrightarrow$ put r in the cover;
- cutting a middle edge $\ell \rightarrow r$: two candidates, ℓ or r , and nothing in the cut says which. Either works, but the association is no longer forced.

KÖNIG VIA MIN CUT

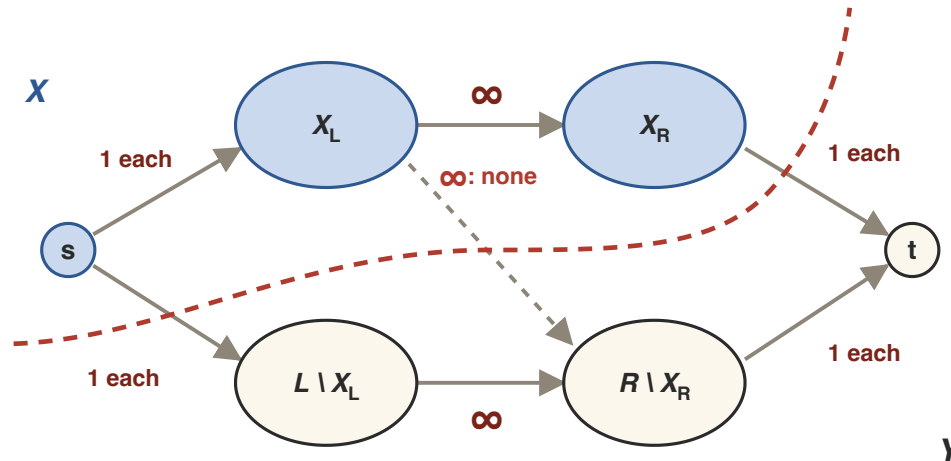
We would like the association to be forced, and exact.

THE ROLE OF ∞ Give the middle edges capacity ∞ . A cut using one costs ∞ while $\{s\}$ costs $|L|$, so a minimum cut uses only s - and t -edges. Each of these is associated with one vertex, so the cut maps to a set of vertices of size exactly its capacity.

The max flow is unchanged: each ℓ still receives at most 1 unit.

KÖNIG VIA MIN CUT

Take a minimum cut $X \ni s$ of the network with ∞ middle edges; let $X_L = X \cap L$ and $X_R = X \cap R$. For $S \subseteq L$, write $N(S)$ for its set of neighbours in R .

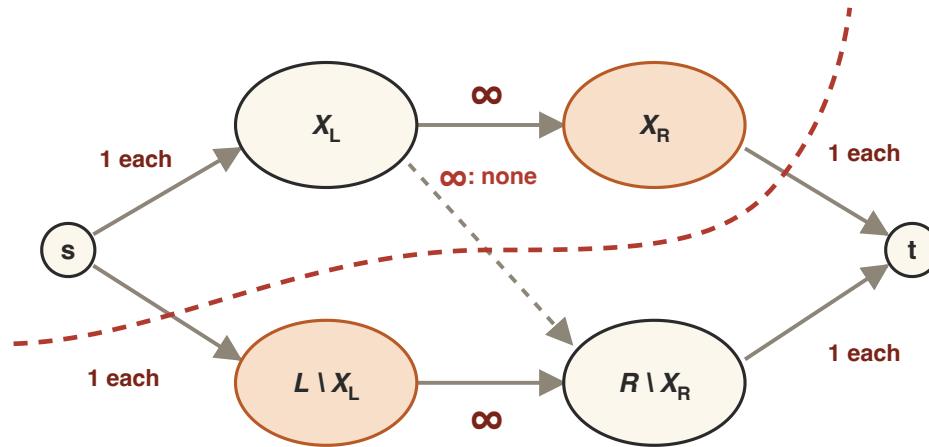


The cut $\{s\}$ has capacity $|L|$, so a minimum cut is finite: no ∞ edge leaves X , i.e.

$$N(X_L) \subseteq X_R.$$

KÖNIG VIA MIN CUT

Let $C = (L \setminus X_L) \cup X_R$: the vertices associated with the crossing edges.



The orange sets form C , one vertex per crossing edge, so $|C|$ equals the cut capacity,
 $|L \setminus X_L| + |X_R|$.

C is a vertex cover: an edge missing C would run from X_L to $R \setminus X_R$ — and there are none.

KÖNIG VIA MIN CUT

A minimum cut X gave a vertex cover C with $|C| = \text{capacity of } X$.

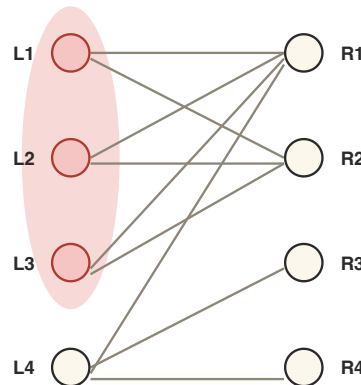
CONCLUSION

$$\underbrace{\tau}_{\text{min vertex cover}} \leq |C| = \text{min cut} = \text{max flow} = \underbrace{\nu}_{\text{max matching}}$$

With weak duality $\nu \leq \tau$, thus $\nu = \tau$. ■

PERFECT MATCHINGS AND AN OBSTACLE

When does a bipartite graph with $|L| = |R| = n$ have a *perfect* matching? Our example has none:



$\{L1, L2, L3\}$ has only two neighbours, $\{R1, R2\}$: three vertices cannot all be matched into two.

In general: a perfect matching forces $|N(S)| \geq |S|$ for every $S \subseteq L$, since S must be matched into $N(S)$.

HALL'S THEOREM

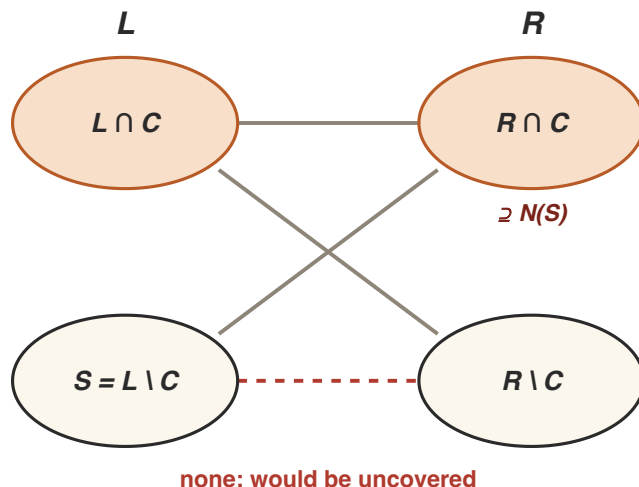
THEOREM (HALL) A bipartite graph with $|L| = |R| = n$ has a perfect matching **if and only if** $|N(S)| \geq |S|$ for every $S \subseteq L$.

A set with $|N(S)| < |S|$ is the *only* obstacle to a perfect matching.

We just saw necessity. Sufficiency is a corollary of König: we show every vertex cover has at least n vertices.

HALL'S THEOREM: A COROLLARY

Assume $|N(S)| \geq |S|$ for all $S \subseteq L$. Let C be a minimum vertex cover (orange) and $S = L \setminus C$, the uncovered part of L .



An edge from S to $R \setminus C$ would have no endpoint in C . So every neighbour of S is covered:
 $N(S) \subseteq R \cap C$.

HALL'S THEOREM: A COROLLARY

Now count C , column by column:

- $|L \cap C| = n - |S|$, since S is the rest of L ;
- $|R \cap C| \geq |N(S)| \geq |S|$, by the picture and by Hall's condition.

So $|C| \geq (n - |S|) + |S| = n$: **every vertex cover has at least n vertices**. And L itself is a cover of size n , so $\tau = n$.

CONCLUSION By König, $\nu = \tau = n$, thus there is a perfect matching. ■

APPLICATION 2: MATRIX ROUNDING

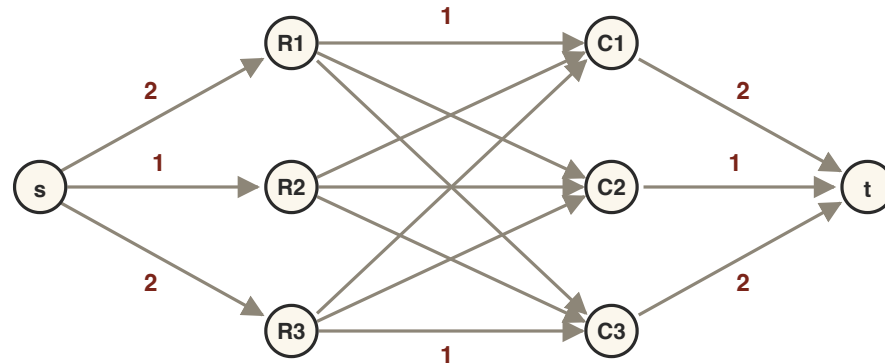
Back to the puzzle: round each entry to an integer, preserving all row and column sums.

Model it as a flow network:

- $s \rightarrow R_i$ with capacity = row sum i ;
- $R_i \rightarrow C_j$ with capacity $\lceil a_{ij} \rceil = 1$ for every entry;
- $C_j \rightarrow t$ with capacity = column sum j .

0.7	0.8	0.5	2
0.3	0.1	0.6	1
1	0.1	0.9	2
2	1	2	

APPLICATION 2: MATRIX ROUNDING



All nine middle edges have capacity 1 (labels shown on two).

MATRIX ROUNDING VIA INTEGRALITY

- The fractional entries themselves are a flow: put a_{ij} on edge $R_i \rightarrow C_j$. Conservation at R_i / C_j is exactly the row / column sum. Value = $0.7 + 0.8 + \dots = 5$, the total sum.
- The cut $X = \{s\}$ has capacity $2 + 1 + 2 = 5$, so this flow is **maximum**.
- All capacities are integers $\Rightarrow$ there is an **integral** max flow of value 5: each middle edge carries 0 or 1.
- Those 0/1 values are the rounded matrix — row and column sums are forced by saturation of the s - and t -edges. The rounding always exists.

MATRIX ROUNDING VIA INTEGRALITY

ONE ANSWER Ford-Fulkerson on this network outputs, e.g., the rounding shown right — check the sums!

1	1	0	2
0	0	1	1
1	0	1	2
2	1	2	

SUMMARY

- **Max flow:** route as much as possible from s to t under capacities; greedy path-packing fails.
- **Residual networks** let augmenting paths undo earlier decisions; Ford-Fulkerson repeats until no augmenting path remains.
- **Max-flow = min-cut:** no augmenting path $\Rightarrow$ the reachable set of G_f is a cut of capacity $|f|$. Cuts certify optimality.

SUMMARY

- **Edmonds-Karp:** BFS-shortest augmenting paths give $O(|V| \cdot |E|^2)$, independent of capacities.
- **Integrality** turns flows into combinatorics: bipartite matching, König's and Hall's theorems, matrix rounding.
- Week 10: the same min-cut theorem falls out of **LP duality**.